

Q36 – Case Study: Medical Imaging Data Sorting (Merge Sort)

Student Details

Course: Design and Analysis of Algorithms

Assignment: CO3_AT2

Algorithm: Merge Sort

Aim

To implement the Merge Sort algorithm for sorting medical imaging patient scan records based on priority and analyze its stability, efficiency, and suitability for large datasets.

Problem Statement

A medical imaging system stores thousands of patient scan records. Emergency cases must be processed before routine scans. The records are sorted according to patient priority while preserving the order of patients having the same priority.

Merge Sort is used because it is a stable sorting algorithm and performs efficiently on large datasets.

Algorithm

1. Divide the list into two equal halves.
2. Continue dividing until each sublist contains one element.
3. Merge adjacent sublists in sorted order.

4. During merging, if two records have equal priority, retain their original order.
 5. Continue merging until one sorted list is obtained.
-

Divide and Conquer Concept

Merge Sort follows the Divide and Conquer strategy.

Divide

- Split the array into two halves recursively.

Conquer

- Sort each half independently.

Combine

- Merge the sorted halves into one sorted array.
-

Stability Analysis

Merge Sort is a stable sorting algorithm.

If two patient records have the same priority, Merge Sort preserves their original sequence.

Example:

Before Sorting

Patient 102 → Priority 5

Patient 104 → Priority 5

After Sorting

Patient 102 → Priority 5

Patient 104 → Priority 5

The relative order remains unchanged.

Issues in Medical Imaging Data

Large Data Size

Hospitals generate thousands of imaging records every day. Efficient sorting is necessary for quick retrieval.

Accuracy

Incorrect ordering may delay emergency treatment. Stable sorting maintains consistent ordering for equal-priority records.

Reliability

Merge Sort guarantees predictable performance even for very large datasets.

Time Complexity

Case	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

Auxiliary Space: $O(n)$

Advantages

- Stable sorting algorithm.

- Suitable for large datasets.
 - Predictable performance.
 - Efficient Divide and Conquer approach.
 - Ideal for healthcare data management.
-

Source Code

```
#include <stdio.h>

typedef struct
{
    int patientID;
    int priority;
    int scanTime;
} ScanRecord;

void merge(ScanRecord arr[], int left, int mid, int right)
{
    int i, j, k;

    int n1 = mid - left + 1;
    int n2 = right - mid;

    ScanRecord L[n1], R[n2];

    for(i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for(j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;

    while(i < n1 && j < n2)
    {
        if(L[i].priority >= R[j].priority)
        {
            arr[k] = L[i];
            i++;
        }
    }
}
```

```

        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while(i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }

    while(j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(ScanRecord arr[], int left, int right)
{
    if(left < right)
    {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

void display(ScanRecord arr[], int n)
{
    int i;

    printf("\nPatientID\tPriority\tScanTime\n");

    for(i = 0; i < n; i++)
    {
        printf("%d\t\t%d\t\t%d\n",

```

```

        arr[i].patientID,
        arr[i].priority,
        arr[i].scanTime);
    }
}

int main()
{
    ScanRecord records[] =
    {
        {101,2,35},
        {102,5,20},
        {103,3,15},
        {104,5,18},
        {105,1,45},
        {106,4,22}
    };

    int n = sizeof(records)/sizeof(records[0]);

    printf("Original Patient Scan Records\n");
    display(records,n);

    mergeSort(records,0,n-1);

    printf("\nSorted Patient Scan Records (Highest Priority
First)\n");
    display(records,n);

    return 0;
}

```

Implementation

main.c

Share

Run

```
1 #include <stdio.h>
2
3 typedef struct
4 {
5     int patientID;
6     int priority;
7     int scanTime;
8 } ScanRecord;
9
10 void merge(ScanRecord arr[], int left, int mid, int right)
11 {
12     int i, j, k;
13
14     int n1 = mid - left + 1;
15     int n2 = right - mid;
16
17     ScanRecord L[n1], R[n2];
18
19     for(i = 0; i < n1; i++)
20         L[i] = arr[left + i];
21
22     for(j = 0; j < n2; j++)
23         R[j] = arr[mid + 1 + j];
24 }
```

Output

Clear

Original Patient Scan Records

PatientID	Priority	ScanTime
101	2	35
102	5	20
103	3	15
104	5	18
105	1	45
106	4	22

Sorted Patient Scan Records (Highest Priority First)

PatientID	Priority	ScanTime
102	5	20
104	5	18
106	4	22
103	3	15
101	2	35
105	1	45

=== Code Execution Successful ===

Sample Input

Patient ID	Priority	Scan Time
101	2	35
102	5	20
103	3	15
104	5	18
105	1	45
106	4	22

Sample Output

Original Patient Scan Records

101 2 35

102 5 20

103 3 15

104 5 18

105 1 45

106 4 22

Sorted Patient Scan Records

102 5 20

104 5 18

106 4 22

103 3 15

101 2 35

105 1 45

Time Complexity:

Best: $O(n \log n)$

Average: $O(n \log n)$

Worst: $O(n \log n)$

Auxiliary Space: $O(n)$

Analysis

The program successfully sorts patient scan records according to priority using Merge Sort. Since Merge Sort is stable, records with equal priority maintain their original order. Its $O(n \log n)$ complexity makes it highly suitable for processing large medical imaging datasets efficiently.

Result

The Merge Sort algorithm was successfully implemented and executed. The patient scan records were sorted correctly while maintaining stability, demonstrating that Merge Sort is an efficient and reliable algorithm for medical imaging data management.